

Cricket Analytics SaaS

Step-by-Step Implementation Blueprint

T20 Match Outcome Prediction Platform
2026 ICC Men's T20 World Cup Edition

8 Phases · 20–24 Weeks · Scratch to Production

Phase	Title	Duration
1	Project Setup & Environment	Week 1–2
2	Data Ingestion & Kafka Streaming	Week 2–4
3	Medallion Data Warehouse (Silver → Gold)	Week 4–7
4	Backend API (FastAPI)	Week 7–9
5	ML Pipeline (XGBoost + Feature Engineering)	Week 9–13
6	GenAI — Agentic RAG + Text-to-SQL	Week 13–16
7	Frontend Dashboards (Next.js)	Week 16–20
8	Deployment, Security & Monetization	Week 20–24

1

Phase 1: Project Setup & Environment

Week 1–2 · Foundation · DevOps · Local Environment

Bootstrap your dev environment, repo structure, and all tooling before writing a single line of application code. A solid foundation here prevents painful refactors later.

1.1 Initialize monorepo structure

- Create a Git monorepo with three root directories: /backend, /frontend, /infra
- Initialize Python 3.11+ virtual environment inside /backend using uv or poetry for dependency management
- Initialize Node.js 20+ project inside /frontend with pnpm
- Add a root .gitignore covering .env files, __pycache__, node_modules, *.pyc, .DS_Store
- Set up pre-commit hooks using the pre-commit library: black + isort for Python, prettier + eslint for JS

Tech stack: [Git](#) · [Python 3.11](#) · [Node 20](#) · [uv/poetry](#) · [pnpm](#) · [pre-commit](#)

✓ Deliverable: Clean repo with enforced code style from day one

1.2 Docker Compose local stack

- Write docker-compose.yml in /infra defining all local services: postgres, redis, kafka, zookeeper, kafka-ui
- Use named volumes for Postgres and Redis data so containers can restart without losing data
- Expose Kafka on 9092, Postgres on 5432, Redis on 6379, Kafka-UI on 8080
- Add a Makefile at root with targets: make up, make down, make logs, make shell
- Test: run make up and verify all containers are healthy via docker ps

Tech stack: [Docker](#) · [Docker Compose](#) · [Kafka](#) · [Zookeeper](#) · [PostgreSQL](#) · [Redis](#)

✓ Deliverable: One-command local environment: make up

1.3 Environment & secrets management

- Create .env.example files in /backend and /frontend listing all required vars — never commit actual .env files
- Required variables: DATABASE_URL, REDIS_URL, KAFKA_BOOTSTRAP_SERVERS, ANTHROPIC_API_KEY, SECRET_KEY, JWT_SECRET
- Install python-dotenv in backend; use Next.js built-in env handling in frontend
- For production: plan to use AWS Secrets Manager or Doppler — document this in README now
- Add GitHub Actions workflow skeleton: .github/workflows/ci.yml with a placeholder test step

Tech stack: [dotenv](#) · [GitHub Actions](#) · [AWS Secrets Manager](#)

✓ Deliverable: .env.example files + CI pipeline skeleton ready

2

Phase 2: Data Ingestion & Kafka Streaming

Week 2–4 · Data Engineering · Streaming · Kafka

Build the data acquisition layer — download Cricsheet data, parse it, and stream it through Kafka to simulate real-time ball-by-ball feeds as if a live match were in progress.

2.1 Download & parse Cricsheet data

- Write scripts/download_cricsheet.py: use requests to fetch the Cricsheet bulk ZIP, extract all JSON files into /data/raw/
- Parse Cricsheet JSON format: extract three sections — meta (version), info (venue, teams, toss, players), innings (ball-by-ball deliveries array)
- Write a CricsheetParser class that flattens nested delivery objects into row dicts: {match_id, inning, over, ball, batter, bowler, runs_off_bat, extras, is_wicket, dismissal_kind}
- Handle null semantics: extras=null → 0, wicket=null → False. Validate: max 10 wickets, max 20 overs per innings
- Unit test the parser with 5 sample match JSONs covering edge cases: super overs, no-balls, retired hurt

Tech stack: [Python](#) · [requests](#) · [json](#) · [dataclasses](#) · [pytest](#)

✓ Deliverable: CricsheetParser class with all unit tests passing

2.2 Build Kafka producer (live simulator)

- Create backend/ingestion/producer.py using kafka-python or confluent-kafka library
- Implement a generator pattern: yield one delivery dict at a time — never load all matches into memory at once
- Add configurable DELIVERY_INTERVAL_MS env var (default 500ms for dev, 45000ms for live simulation mode)
- Inject an event_timestamp field (datetime.utcnow().isoformat()) to each message before publishing — critical for event-time processing
- Publish to Kafka topic 'cricket.deliveries' using match_id as partition key so all balls of a match go to the same partition
- Add a second topic 'cricket.match_events' for match-level events: match_start, innings_break, match_end

Tech stack: [kafka-python](#) · [confluent-kafka](#) · [Python generators](#)

✓ Deliverable: Producer script streaming 1 delivery/500ms to Kafka topic

2.3 Build Kafka consumer & Bronze layer sink

→ Create backend/ingestion/consumer.py: subscribe to 'cricket.deliveries', commit offsets manually for at-least-once delivery guarantees

→ Sink raw JSON payloads to /data/bronze/ partitioned by date:

bronze/year=2024/month=01/day=15/batch_001.json

→ Write to bronze_deliveries table in PostgreSQL with columns: raw_payload JSONB, kafka_offset BIGINT, ingested_at TIMESTAMP

→ Implement dead-letter queue: messages failing JSON validation go to topic 'cricket.deliveries.dlq' with error metadata

→ Integration test: run producer + consumer simultaneously, verify bronze table row count matches producer message count

Tech stack: [kafka-python](#) · [PostgreSQL](#) · [JSONB](#) · [psycopg2](#)

✓ Deliverable: End-to-end pipeline: producer → Kafka → consumer → Bronze PostgreSQL table

3

Phase 3: Medallion Data Warehouse (Silver → Gold)

Week 4–7 · Data Warehouse · dbt · Star Schema · ETL · Multi-tenancy

Transform raw Bronze JSONB into clean Silver tables, then build the Star Schema Gold layer with fact and dimension tables that power all analytics and ML. Implement Row-Level Security for multi-tenancy.

3.1 Silver layer: clean & flatten deliveries

- Install dbt-postgres. Run dbt init cricket_analytics inside /backend/dbt/
- Create models/silver/stg_deliveries.sql: SELECT from bronze_deliveries, use jsonb_extract_path_text() to pull fields, CAST to correct types
- Apply: COALESCE(extras, 0), CAST(is_wicket AS BOOLEAN), UPPER(dismissal_kind), standardize player names via seed CSV mapping Cricsheet IDs to global IDs
- Add dbt tests in schema.yml: not_null on match_id/batter/bowler, accepted_values for dismissal_kind, relationships between tables
- Create stg_match_info.sql: one row per match with venue, toss_winner, toss_decision, match_date, tournament_name, teams

Tech stack: [dbt](#) · [PostgreSQL](#) · [SQL](#) · [JSONB](#)

✓ Deliverable: Silver layer: clean stg_deliveries and stg_match_info tables

3.2 Gold layer: dimension tables

- dim_player.sql: SELECT DISTINCT player data, add surrogate key via dbt_utils.generate_surrogate_key(), include batting_style, bowling_style, national_team. Implement SCD Type 2 via dbt snapshots
- dim_venue.sql: stadium_name, city, country, pitch_type (Spin/Pace/Flat/Batting), average_first_innings_score, boundary_size_category
- dim_match_context.sql: tournament_phase (Group/Super8/Semifinal/Final), toss_winner_key, toss_decision, weather_category, day_night_flag
- dim_date.sql: standard date spine using dbt_utils.date_spine(), add: day_of_week, is_weekend, tournament_week_number
- Seed dim_venue_manual.csv with manually curated pitch data for all 2026 World Cup venues (Ahmedabad, Colombo, Wankhede, Eden Gardens, etc.)

Tech stack: [dbt](#) · [dbt_utils](#) · [SCD Type 2](#) · [dbt snapshots](#)

✓ Deliverable: 4 dimension tables in Gold layer with surrogate keys

3.3 Gold layer: fact tables

→ fact_delivery.sql: one row per ball. Columns: delivery_sk, match_id, over_number, ball_number, batter_key (FK→dim_player), bowler_key (FK→dim_player), venue_key (FK→dim_venue), date_key (FK→dim_date), runs_off_bat, extras, is_wicket, boundary_flag, is_dot_ball, tenant_id

→ fact_match_summary.sql: one row per match, aggregated from fact_delivery — total_runs, wickets, winner_team_key, margin_runs, margin_wickets, toss_winner_won

→ Add win_probability_delta FLOAT column to fact_delivery (initially NULL, populated by ML pipeline in Phase 5)

→ Run dbt build --select gold.* and verify: SUM(runs_off_bat) in fact_delivery must equal total_runs in fact_match_summary per match

→ Add the dbt elementary package for automated data quality monitoring with an observability dashboard

Tech stack: [dbt](#) · [PostgreSQL](#) · [elementary](#) · [dbt tests](#)

✓ Deliverable: Complete Star Schema: 2 fact tables + 4 dimension tables with referential integrity

3.4 Multi-tenancy: Row-Level Security

→ Add tenant_id VARCHAR column to all Gold layer tables (fact_delivery, fact_match_summary)

→ In PostgreSQL: CREATE POLICY tenant_isolation ON fact_delivery USING (tenant_id = current_setting('app.current_tenant'))

→ Enable RLS: ALTER TABLE fact_delivery ENABLE ROW LEVEL SECURITY

→ In FastAPI: on every DB connection, execute SET app.current_tenant = '{tenant_id}' before any query — enforces isolation at the DB engine level, not application layer

→ Integration test: create two tenants, insert data for each, verify tenant A queries return zero rows from tenant B's data

Tech stack: [PostgreSQL RLS](#) · [Row-Level Security](#) · [FastAPI](#)

✓ Deliverable: Tenant-isolated data warehouse with zero cross-tenant data leakage

4

Phase 4: Backend API (FastAPI)

Week 7–9 · Backend · FastAPI · Auth · REST API · JWT

Build the FastAPI backend that serves as the business logic layer — authentication, tenant management, rate limiting, and all REST API endpoints serving the three user personas.

4.1 FastAPI project structure & core setup

- Directory structure: backend/app/main.py, /routers/, /models/, /schemas/, /dependencies/, /services/, /db/
- Install: fastapi, uvicorn, sqlalchemy 2.0, alembic, pydantic v2, python-jose, passlib, httpx
- Configure SQLAlchemy 2.0 async engine with asyncpg driver pointing to PostgreSQL
- Run alembic init for DB migrations. Create initial migration: users, tenants, subscriptions tables
- Add CORS middleware allowing the Next.js frontend origin, and GZipMiddleware for response compression

Tech stack: [FastAPI](#) · [SQLAlchemy 2.0](#) · [Alembic](#) · [asyncpg](#) · [Pydantic v2](#)

✓ Deliverable: FastAPI app boots, DB migrations run, /health endpoint returns 200

4.2 Authentication & tenant context injection

- Implement JWT authentication: POST /auth/register, POST /auth/login return access_token (15min TTL) + refresh_token (7 days)
- Create get_current_user FastAPI dependency: decodes JWT, fetches user from DB, returns User object with tenant_id and role
- Create get_tenant_context dependency: extracts tenant_id from authenticated user, injects into every DB session via SET app.current_tenant
- Role-based access control: roles = [fan, professional, enterprise, admin]. Decorate endpoints with @require_role(['professional', 'enterprise'])
- Rate limiting via slowapi: Fan tier = 100 req/hour, Professional = 1000/hour, Enterprise = unlimited

Tech stack: [JWT](#) · [python-jose](#) · [passlib](#) · [slowapi](#)

✓ Deliverable: Auth endpoints working, tenant isolation enforced on every DB query

4.3 Core analytics API endpoints

- GET /matches — paginated list with filters: team, venue, date_range, tournament_phase
- GET /matches/{id}/win-probability — fetch from Redis cache first, fallback to ML inference service
- GET /players/{id}/stats — rolling averages (last 10 innings), phase-wise strike rates (Powerplay/Middle/Death), head-to-head matchup stats
- GET /venues/{id}/analytics — average scores, toss impact, pitch behavior, historical chase success rate
- GET /tournaments/{id}/leaderboard — team ELO rankings, top batters/bowlers by phase
- All endpoints return Pydantic v2 response models with proper type hints. Full OpenAPI docs at /docs

Tech stack: [FastAPI routers](#) · [Redis](#) · [Pydantic v2](#) · [OpenAPI](#)

✓ Deliverable: 6 core endpoints documented at /docs and returning correct data

5

Phase 5: ML Pipeline (XGBoost + Feature Engineering)

Week 9–13 · Machine Learning · XGBoost · SHAP · MLflow · Ray Serve

Build the feature engineering pipeline, train the XGBoost win-probability model, integrate SHAP for interpretability, and deploy via MLflow + Ray Serve for sub-200ms inference.

5.1 Feature engineering pipeline

- Create backend/ml/features.py with a FeatureEngineer class. Input: all deliveries up to ball N. Output: 25-dimensional feature vector
- Dynamic match state features: balls_remaining, wickets_in_hand, current_run_rate (CRR), required_run_rate (RRR), run_rate_differential (CRR - RRR), run_lead_or_deficit
- Player strength features: batter_rolling_sr_last10 (strike rate over last 10 innings), bowler_rolling_economy_last10, team_elo_score (sum of playing XI individual ELO ratings)
- Contextual features: venue_avg_first_innings (from dim_venue), toss_venue_impact (interaction: toss_decision x venue_chase_success_rate), pitch_phase (NEW_BALL/POWERPLAY_SPIN/DEATH), is_knockout boolean
- Unit tests: given a mock match state, assert feature vector has correct shape and zero NaN values

Tech stack: Python · pandas · numpy · scikit-learn

✓ Deliverable: FeatureEngineer class producing a 25-feature vector per delivery

5.2 Train & evaluate XGBoost model

- Load fact_delivery from PostgreSQL via SQLAlchemy. Build training set: feature vectors for every delivery. Target: match_winner (binary 0/1 for team batting first)
- Time-based train/val/test split: before 2023 = train, 2023 = val, 2024 = test. Never shuffle — future data must not leak into training
- Train with xgboost.XGBClassifier, optimize log-loss. Use Optuna for hyperparameter search: max_depth, learning_rate, n_estimators, subsample, colsample_bytree
- Evaluate: log-loss on test set, calibration curve (a 70% prediction should win ~70% of the time), Brier score
- Use MLflow to track every experiment: log params, metrics, and the model artifact. Run mlflow ui to inspect experiments

Tech stack: XGBoost · Optuna · MLflow · scikit-learn · pandas

✓ Deliverable: Trained model artifact in MLflow with log-loss < 0.45 on test set

5.3 SHAP interpretability & real-time inference

- Add `shap.TreeExplainer` to the inference pipeline. For each prediction, compute SHAP values and return top 3 contributing features with impact magnitude and direction
- Example output: `{win_probability: 0.72, explanation: [{feature: 'wickets_in_hand', impact: -0.18, direction: 'negative'}, ...]}`
- Build `backend/ml/inference.py`: load MLflow model artifact, accept `match_state` dict, run `FeatureEngineer`, return `prob` + SHAP explanation
- Integrate with Redis: after each Kafka delivery event, run inference, cache result in Redis key `'wp:{match_id}'` with 120s TTL
- Deploy as a Ray Serve endpoint on `localhost:8000/predict` — FastAPI calls this internal service, not the model directly

Tech stack: SHAP · Ray Serve · MLflow · Redis · XGBoost

✓ Deliverable: Win probability API: POST `/predict` returns `prob` + SHAP explanation in <200ms

5.4 Additional ML use cases

- Score forecasting (regression): `XGBoostRegressor` trained on Powerplay state (6 overs) → predict final score. Features: `pp_runs`, `pp_wickets`, `venue_avg`, `team_batting_elo`
- Player clustering (K-Means): cluster players on `[avg_sr, avg_economy, dot_ball_pct, boundary_pct, death_over_sr]`. K=5 archetypes: Anchor, Finisher, Powerplay Specialist, Death Bowler, All-Rounder
- Matchup association rules (Apriori via `mlxtend`): mine frequent `{bowler_style → batter_dismissal}` patterns. Expose via GET `/matchups/vulnerabilities`
- Fantasy points projection: weighted formula = $(\text{expected_runs} \times 1) + (\text{expected_wickets} \times 25) + (\text{catch_prob} \times 8)$. Expose via GET `/players/{id}/fantasy-projection`

Tech stack: XGBoost Regressor · scikit-learn KMeans · mlxtend Apriori

✓ Deliverable: 4 ML models deployed: classifier, regressor, clustering, association rules

6

Phase 6: GenAI — Agentic RAG + Text-to-SQL

Week 13–16 · GenAI · LangGraph · RAG · LLM · ChromaDB · Text-to-SQL

Integrate the Agentic RAG system using LangGraph — routing natural language queries to either Text-to-SQL for structured stats or vector search for unstructured pitch reports and commentary.

6.1 Text-to-SQL agent tool

- Install: langchain, langgraph, langchain-anthropic, anthropic
- Build a schema context string: a detailed description of all Gold layer tables, column names, data types, and relationships — injected into LLM system prompt
- Create tools/text_to_sql.py: a LangChain tool that accepts NL query, calls Claude with schema context, receives SQL, sanitizes (no DROP/DELETE/UPDATE), executes in READ-ONLY transaction, returns JSON
- Add tenant_id injection: before executing any SQL, append AND tenant_id = '{tenant_id}' to every WHERE clause programmatically
- Test: 'What is the average first innings score at Wankhede?' → correct SQL → correct numerical answer

Tech stack: LangChain · LangGraph · Claude API · SQLAlchemy

✓ Deliverable: Text-to-SQL tool: natural language query → correct SQL → numerical answer

6.2 Vector search tool for unstructured data

- Collect unstructured documents: pitch curator reports, umpire match reports, expert commentary. Store as .txt in /data/documents/
- Install: chromadb, langchain-community
- Chunk documents by paragraph (max 500 tokens). Generate embeddings using claude-3-haiku via Anthropic API. Store vectors + metadata in ChromaDB collection 'cricket_docs'
- Create tools/vector_search.py: accepts a query string, embeds it, runs ChromaDB similarity search (top-5 chunks), returns retrieved context
- Test: 'What were pitch conditions in Colombo last tournament?' → relevant curator report chunks returned

Tech stack: ChromaDB · LangChain · Anthropic Embeddings · Claude Haiku

✓ Deliverable: Vector store populated, similarity search returning relevant documents

6.3 LangGraph agentic router

- Build a LangGraph StateGraph with nodes: router_node, text_to_sql_node, vector_search_node, synthesizer_node
- Router node: Claude analyzes query and decides — specific numbers/records/stats → text_to_sql; qualitative analysis/summaries → vector_search; both types → invoke both in parallel
- Synthesizer node: combines tool outputs, passes to Claude for final natural language answer. System prompt: 'Never hallucinate statistics. If SQL returned no results, say so.'
- Expose via POST /ai/chat in FastAPI. Accept {query, conversation_history}. Return {answer, sources, tool_used}
- Add conversation memory: store last 5 turns in Redis keyed by session_id to enable follow-up questions

Tech stack: [LangGraph](#) · [LangChain](#) · [Redis](#) · [FastAPI](#)

✓ Deliverable: Conversational AI endpoint routing to correct tool and returning grounded answers

6.4 Dynamic narrative generation

- Create NarrativeService in backend/services/narrative.py
- Subscribe to Redis pub/sub channel 'match.probability.updated'. When win_probability_delta > 0.05 (significant swing), trigger narrative generation
- Prompt: 'Given this match state and a win probability swing of {delta}, write 2-sentence broadcast-style commentary for {audience_type}.' Audience: coach (tactical), bettor (financial), broadcaster (dramatic)
- Cache narratives in Redis: 'narrative:{match_id}:{over}' with 300s TTL
- Expose via GET /matches/{id}/narrative?persona=broadcaster

Tech stack: [Claude API](#) · [Redis Pub/Sub](#) · [LangChain](#)

✓ Deliverable: Auto-generated momentum narratives triggered by significant probability swings

7

Phase 7: Frontend Dashboards (Next.js)

Week 16–20 · Frontend · Next.js · React · Recharts · WebSocket · Tailwind

Build three persona-specific dashboards (Coach, Bettor, Broadcaster) plus the streaming AI chat interface, using Next.js 14 with App Router and Recharts for data visualizations.

7.1 Next.js app setup & shared components

- Initialize: `npx create-next-app@latest` with App Router, TypeScript, Tailwind CSS
- Install: `recharts`, `@tanstack/react-query`, `axios`, `zustand`, `socket.io-client`, `shadcn/ui`
- Build shared components: Navbar (logo, user menu, tier badge), StatCard (metric + trend), WinProbabilityGauge (semi-circle, live WebSocket updates), PersonaBadge
- Set up React Query for all API calls with 30s stale time. Configure axios interceptor to attach JWT Bearer token to every request automatically
- Build AuthContext with Zustand: stores user, `tenant_id`, `subscription_tier`. Implement login/logout flows

Tech stack: [Next.js 14](#) · [TypeScript](#) · [Tailwind](#) · [React Query](#) · [Zustand](#) · [shadcn/ui](#)

✓ Deliverable: Shared component library + auth flow working end-to-end

7.2 Coach dashboard (/dashboard/coach)

- Gate with `@require_role(['professional', 'enterprise'])`
- Spray chart: ScatterChart (Recharts) showing where a batter hits the ball. X/Y from delivery angle data. Color-coded by shot type
- Phase-wise strike rate chart: BarChart with 3 bars per player — Powerplay SR, Middle Overs SR, Death SR. Tooltip shows dismissal count per phase
- Bowler matchup heatmap: table with batters as rows, bowler styles as columns, cells show dismissal rate. Color scale: red (vulnerable) → green (dominant)
- Playing XI optimizer: input opponent team, get algorithmic recommendation for optimal batting order based on head-to-head data

Tech stack: [Recharts](#) · [Next.js](#) · [TanStack Query](#)

✓ Deliverable: Coach dashboard with spray chart, phase analysis, matchup heatmap

7.3 Bettor & broadcaster dashboards

- Bettor dashboard (/dashboard/bettor): Live WinProbabilityGauge updating via WebSocket every ball. Historical odds trendline (last 20 overs). Fantasy Dream XI with projected points. Situational Pressure Index gauge per key player
- Broadcaster dashboard (/dashboard/broadcaster): Big live win probability with dramatic color animation on swings. Momentum shift detector: over-by-over probability timeline with annotated turning points. Real-time milestone alerts and auto-generated narrative cards
- WebSocket integration: connect to FastAPI WebSocket at /ws/match/{match_id}. On each delivery receive {win_prob, narrative, last_delivery_summary} and update all reactive components
- Implement subdomain routing in Next.js middleware: teama.app.com → inject tenant_id='teama' into all API requests automatically

Tech stack: [WebSocket](#) · [Recharts](#) · [Next.js Middleware](#)

✓ Deliverable: All 3 persona dashboards live with real-time win probability updates

7.4 AI chat interface (/dashboard/ai-chat)

- Chat UI with message history, typing indicator, and source citation display below each AI response
- Each response shows: answer text, 'Tool used' badge (Text-to-SQL or Vector Search), clickable source links
- Quick-action prompt chips: 'Best chaser at this venue', 'Compare teams head-to-head', 'Predicted score from Powerplay'
- Streaming responses: use fetch with ReadableStream to display Claude's answer word-by-word as it generates (FastAPI Server-Sent Events)
- Gate full AI chat behind Professional/Enterprise tier. Fan tier gets 5 queries/day with an upgrade paywall modal

Tech stack: [Server-Sent Events](#) · [ReadableStream](#) · [React](#)

✓ Deliverable: Streaming AI chat interface with tool attribution and tier gating

8

Phase 8: Deployment, Security & Monetization

Week 20–24 · DevOps · Docker · Kubernetes · AWS · Stripe · Security · Monitoring

Containerize all services, deploy to production on AWS with Kubernetes, implement the Stripe subscription payment system, and harden security for a commercial-grade launch.

8.1 Dockerize all services

- FastAPI Dockerfile: python:3.11-slim base, copy requirements, run as non-root user (USER 1000), EXPOSE 8000
- Next.js Dockerfile: node:20-alpine, npm install --frozen-lockfile, next build, EXPOSE 3000
- ML inference Dockerfile (Ray Serve): python:3.11-slim, install ray[serve] xgboost shap, copy model artifact from MLflow
- docker-compose.prod.yml: all services with resource limits (backend: 512MB RAM, ml-inference: 2GB RAM), health checks, restart: unless-stopped
- NGINX reverse proxy container: SSL termination, subdomain routing (teama.cricketai.com → backend with X-Tenant-ID header), rate limiting at proxy level

[Tech stack: Docker · NGINX · Docker Compose](#)

✓ Deliverable: All services containerized, docker-compose.prod.yml boots full stack

8.2 Kubernetes production deployment (AWS EKS)

- Write Kubernetes manifests in /infra/k8s/: Deployment, Service, Ingress, HorizontalPodAutoscaler for each service
- Backend: min 2 replicas, max 10. ML inference: min 1, max 5. Scale on CPU > 70%
- Use AWS RDS PostgreSQL (Multi-AZ) for production DB. AWS ElastiCache Redis. Amazon MSK (managed Kafka) replacing containerized Kafka
- Configure cert-manager for automatic SSL certificates via Let's Encrypt. Enforce HTTPS on all traffic
- GitHub Actions CI/CD: on push to main → run tests → build Docker images → push to ECR → kubectl rollout restart

[Tech stack: Kubernetes · AWS EKS · AWS RDS · ElastiCache · Amazon MSK · cert-manager](#)

✓ Deliverable: Production cluster on AWS with auto-scaling and zero-downtime deploys

8.3 Subscription & payment system (Stripe)

- Integrate Stripe: create 3 products — Fan Free, Professional \$25/month, Enterprise (custom pricing)
- Webhook handler at POST /stripe/webhook: on checkout.session.completed → upgrade user subscription_tier in DB. On invoice.payment_failed → downgrade to Fan
- Build /pricing page in Next.js with Stripe Checkout redirect for Professional tier signup
- Implement usage metering for Enterprise API: count API calls per tenant per day in Redis. Bill overages via Stripe Metered Billing
- Add Stripe Customer Portal link in user settings for self-serve subscription management (upgrade, downgrade, cancel)

Tech stack: [Stripe](#) · [Stripe Webhooks](#) · [Stripe Metered Billing](#)

✓ Deliverable: Working payment flow: fan → professional upgrade via Stripe Checkout

8.4 Security hardening & monitoring

- SQL injection prevention: all queries use parameterized statements via SQLAlchemy ORM — never string interpolation
- Input validation: all API inputs validated via Pydantic v2 strict mode. Add max_length limits on all string fields
- Sentry for error tracking in FastAPI and Next.js. Alert on error rate > 1%
- Grafana + Prometheus: instrument FastAPI with prometheus-fastapi-instrumentator. Monitor: request latency p95, Kafka consumer lag, Redis cache hit rate, ML inference latency
- Audit logging: every data access logged to audit_log table with user_id, tenant_id, endpoint, timestamp — required for enterprise compliance
- Run OWASP ZAP security scan before launch. Address all high/critical severity findings before go-live

Tech stack: [Sentry](#) · [Prometheus](#) · [Grafana](#) · [OWASP ZAP](#) · [Pydantic v2](#)

✓ Deliverable: Production-ready: monitoring dashboards live, security audit passed

Critical Implementation Notes

Row-Level Security — implement from day one

The PostgreSQL RLS + SET app.current_tenant injection in FastAPI must be in place before any Gold layer data is loaded. Retrofitting multi-tenancy after the fact is extremely painful. Never skip this in Phase 3.

Time-based train/test split is non-negotiable

For the ML model in Phase 5, always split by date (before 2023 = train, 2024 = test). Never shuffle cricket data. Training on 2024 matches and testing on 2022 gives artificially inflated accuracy that collapses in production.

Kafka generator pattern for memory safety

In the Phase 2 producer, always use Python generators to yield one delivery at a time from Cricsheet JSON files. Loading the full corpus (~7,470 matches) into RAM will crash the process.

SHAP values are a product requirement, not an add-on

Enterprise clients (franchises, broadcasters) need to understand why the model predicted what it did. Integrate SHAP TreeExplainer in Phase 5 alongside the model — not as a post-launch feature.

Subdomain routing must be set up before the first tenant

The NGINX + Next.js middleware subdomain routing (team.cricketai.com → tenant_id injection) should be validated in Phase 8 before onboarding any enterprise client.

Stripe webhooks before going live

Test Stripe webhook handling with the Stripe CLI (stripe listen --forward-to localhost) in development. A failed webhook means a paying customer who gets no access — this is a business-critical path.